```
run -> (env) { [200, {"content-type" => "text/plain"}, ["Welcome to Rack"]] }
```

And here, with a custom object

```
class SmallRackApp
  def call(env)
    [200, {"content-type" => "text/plain"}, ["Welcome to Rack"]]
  end
end

run SmallRackApp.new
```

In all these cases, the callable takes one argument—the *environment*—and returns three-element array as the response. The structure of both the environment and response are defined by the Rack Specification.[12]

The environment is an unfrozen instance of Hash. Because the environment is unfrozen, a Rack app can modify it, often to add new keys. The environment is required to have several specific keys even if sometimes those keys are empty.

Those keys include the URL of the request, which is split into the keys SERVER_NAME, SCRIPT_NAME, PATH_INFO, QUERY_STRING, and SERVER_PORT. The HTTP verb of the request is in the key REQUEST_METHOD. Any HTTP request headers are placed in corresponding keys starting with HTTP_. Rack puts information of whether the request uses HTTP or HTTPS in the key rack.url_scheme and adds a raw input stream at rack.input and an error stream at rack.errors.

The response is a three-element unfrozen array made up of:

- The HTTP status, which must be an integer greater than or equal to 100.
- The headers to be sent as part of the response, as an unfrozen Hash. Keys in this hash that start with rack. are not sent back to the client browser.
- The body of the response. Most commonly, the body is an array of strings.

The body element of the response can be more complicated. Rather than an array, the body can be any Ruby enumerable that responds to the method each and results in a series of strings.

Less commonly, the body can act as something that is streamed, rather than returned all at once. In this case the body is Proc or anything that responds to call, in which case the result of call should be something that behaves like an IO stream, where reading from the stream returns data.

Where this starts to get really powerful is with the ability of these Rack applications to chain together, allowing an application to act on the environment or response of a different application. This is called Rack middleware. And while the middleware uses the same environment and response objects, there's a little bit more structure.

Here's an example:

web/rack_02/config.ru

```
require "bundler/inline"

gemfile do
  source "https://rubygems.org"
```

12. https://github.com/rack/rack/blob/main/SPEC.rdoc